講義 2：變數與 TYPE / LIKE / CONSTANTS

> 對應練習：ex02 | 答案程式：ZR_TR02_TYPE_LIKE

本講重點

- DATA 宣告變數：TYPE、LENGTH、DECIMALS、VALUE
- ABAP 內建資料型別總表與各自的適用場景
- TYPE（參考型別）與 LIKE（參考另一個變數）的差別
- CONSTANTS 常數
- 賦值 =、CLEAR，以及自動型別轉換的行為

1. 宣告變數：DATA

```
DATA 變數名 TYPE 型別 [LENGTH 長度] [DECIMALS 小數位] [VALUE 初始值].
```

```
DATA gv_name  TYPE c LENGTH 10 VALUE 'ABAP'.    " 固定長度文字
DATA gv_count TYPE i VALUE 100.                 " 整數
DATA gv_price TYPE p LENGTH 8 DECIMALS 2.       " 帶小數的金額
DATA gv_memo  TYPE string.                      " 變動長度文字

* 鏈式寫法（實務標準寫法）：
DATA: gv_qty   TYPE i,
      gv_today TYPE d.
```

新式寫法對照（7.40 之後）：行內宣告（Inline Declaration）DATA(...)

傳統寫法是「先宣告、再使用」；7.40 之後可以在**第一次賦值的地方**直接宣告，型別由右邊的值自動推導：

```
* 傳統寫法：先宣告，再賦值
DATA gv_count TYPE i.
gv_count = 3.

* 新式寫法：宣告與賦值合在同一行
DATA(gv_count) = 3.
```

語法拆解：DATA(變數名) 寫在等號**左邊**，意思是「這裡宣告一個新變數」；它的型別不是自己寫的，而是**照抄右邊那個值的型別**。所以右邊放什麼，決定了變數是什麼型別（2026-10-08 用 ZR_TR02_TYPE_LIKE_NEW、ZR_TR18_DATE_CHK 實測）：

寫法	推導出的型別	要注意的地方
DATA(gv_n) = 3.	i	之後 gv_n = 7 / 2. 得 4（整數四捨五入）

寫法	推導出的型別	要注意的地方
`DATA(gv_s) = 'TR02'.`	`c LENGTH 4`	長度固定為 4，之後放更長的文字會被截斷
` `DATA(gv_s) = ABAP.` `	`string`	要變動長度字串就用反引號（講義 1）
`DATA(gv_d) = sy-datum.`	字典型別 `SYDATUM`	連字典型別一起抄過來，所以 `WRITE` 會印成 `2026/10/08`；自己宣告 `TYPE d` 則印 `20261008`（講義 18 §6）
`DATA(gv_p) = gv_price.`	跟 `gv_price` 一樣	效果等於 `LIKE gv_price`

什麼時候還是要先宣告：

- **帶小數的金額**：寫 `DATA(gv_price) = '1250.50'.` 得到的是 `c LENGTH 7`（文字），不是 `p`。金額這類型別無法從字面值（Literal）推導的，仍要 `DATA gv_price TYPE p LENGTH 8 DECIMALS 2.`。
- **同一個名稱只能行內宣告一次**：作用範圍是整個程式單元（程式主體 / FORM / 方法），第二次寫 `DATA(gv_count)` 會編譯錯誤 `"GV_COUNT" was already declared`。

可以用 `DATA( )` 的位置：不只賦值，很多指令「放結果的地方」都能行內宣告。2026-10-08 用 `ZR_TR02_INLINE_CHK` 逐一實測：

位置	寫法	推導出的型別	詳見
賦值左邊	`DATA(gv_count) = 3.`	右邊值的型別	本節
`LOOP ... INTO`	`LOOP AT gt INTO DATA(gs).`	內表（Internal Table）的列型別	講義 4 §4
`READ TABLE ... INTO`	`READ TABLE gt INTO DATA(gs) WITH KEY ...`	內表的列型別	講義 4 §5
`SELECT ... INTO`	`INTO TABLE @DATA(gt)`、`INTO @DATA(gv)`	依 SELECT 清單	講義 6 §3.6
`SPLIT ... INTO`	`SPLIT gv_csv AT ',' INTO DATA(a) DATA(b).`	`string`	講義 18 §2
`CONCATENATE ... INTO`	`CONCATENATE a b INTO DATA(gv_full).`	`string`	講義 18 §1
`FIND` 的計數	`FIND ALL OCCURRENCES OF 'a' IN s MATCH COUNT DATA(gv_cnt).`	`i`	
`REPLACE` 的計數	`REPLACE ALL OCCURRENCES OF ... REPLACEMENT COUNT DATA(gv_cnt).`	`i`	
`DESCRIBE TABLE`	`DESCRIBE TABLE gt LINES DATA(gv_lines).`	`i`	講義 5 §5
`GET TIME STAMP`	`GET TIME STAMP FIELD DATA(gv_ts).`	字典型別 `TZNTSTMPS`	

位置	寫法	推導出的型別	詳見
`LOOP ... GROUP BY`	`GROUP BY ( ... ) INTO DATA(gs_grp)`	群組	講義 20 §5.1
`CATCH` 例外	`CATCH cx_sy_zerodivide INTO DATA(lx_err).`	指向該例外類別的參照	OOP 課程
Field-Symbol	`LOOP AT gt ASSIGNING FIELD-SYMBOL(<fs>).`	內表的列型別	講義 16 §7

不能用的位置（要事先宣告好變數）：

- `WRITE ... TO`：`WRITE sy-datum TO DATA(gv_txt).` 啟用時報 `Field "GV_TXT" is unknown`（實測）。
- `CALL FUNCTION` 的參數：`CALL FUNCTION ... IMPORTING ev_x = DATA(gv_x)` 報 `The inline declaration "DATA(GV_X)" is not possible in this position`（講義 15 會用到 `CALL FUNCTION`）。

`SPLIT`、`CONCATENATE` 推導出的一律是 `string`，即使來源是 `c` 字面值；要固定長度的 `c`，就得事先宣告。

行內宣告只是少寫一行宣告，執行結果與效能跟傳統寫法相同。練習 2 的新式寫法版本見答案程式 `ZR_TR02_TYPE_LIKE_NEW`，輸出跟原版逐行比對過，只有日期一行因為上表的型別差異而格式不同。

命名慣例（本課程與團隊風格一致）：全域變數 `gv_`（value）、`gs_`（structure）、`gt_`（table）；區域變數換成 `lv_` / `ls_` / `lt_`。前綴讓人一眼看出「這是什麼形狀的資料」。

2. 內建資料型別（Built-in Types）總表

型別	名稱	預設長度	初始值	用途與注意
`c`	固定長度字元	1	空白	**不給 LENGTH 就只有 1 個字元**，超長會被截斷
`n`	數字字元	1	'0...0'	存「長得像數字的編號」（如 0001），不能拿來運算
`string`	變動長度字串	-	空字串	長度自動伸縮，一般文字建議用它
`i`	整數	4 bytes	0	計數、筆數
`p`	壓縮十進位	8 bytes	0	**金額、數量的標準型別**，搭配 DECIMALS 精確小數
`f`	浮點數	8 bytes	0.0	科學計算用，有精度誤差，**金額禁用**
`d`	日期	8 字元	'00000000'	格式 YYYYMMDD，可直接加減天數
`t`	時間	6 字元	'000000'	格式 HHMMSS

型別	名稱	預設長度	初始值	用途與注意
`x` / `xstring`	十六進位	-	-	二進位資料（檔案內容），先認識即可

三個最容易踩的點：

- `TYPE c` 忘了 LENGTH：`DATA gv_x TYPE c.` 只能存 1 個字，塞 'ABC' 只剩 'A'。
- 金額用 `f`：浮點有二進位精度誤差（0.1 存不準），金額一律 `p ... DECIMALS n`。
- `n` 型別參與運算：它本質是字元，先轉成 `i`/`p` 再算。

日期運算範例（`d` 可以直接加減）：

```
DATA: gv_today TYPE d,
      gv_due   TYPE d.
gv_today = sy-datum.          " 系統欄位：今天日期
gv_due   = gv_today + 30.     " 30 天後
WRITE: / '今天：', gv_today, '30 天後：', gv_due.
```

3. TYPE 與 LIKE 的差別

寫法	參考對象	讀法
`DATA a TYPE i.`	型別（內建型別、TYPES 定義、DDIC 型別）	「a 的型別是 i」
`DATA b LIKE a.`	**另一個資料物件**（變數、系統欄位（System Field）、表格）	「b 跟 a 長一樣」

```
DATA gv_date1 TYPE d.            " TYPE：參考型別
DATA gv_date2 LIKE sy-datum.     " LIKE：參考既有變數（系統欄位）
DATA gv_date3 LIKE gv_date1.     " LIKE：跟 gv_date1 同型別
```

使用時機：

- 想表達「跟某個既有欄位 / 變數保持一致」用 `LIKE`——來源改了，跟著的變數自動一致。
- 其他情況用 `TYPE`。本講先用內建型別（`i`、`c`、`d`.....）學語法；但**內建型別只適合純區域暫存**（計數器、旗標），只要變數代表業務資料，實務上都是 `TYPE scarr-carrid` 這種參考資料字典（Data Dictionary）欄位的寫法（Global Type，講義 6 第 1.1 節詳解）。
- 進階常用：`DATA gs LIKE LINE OF gt_tab.`（宣告跟某內表一列同型別的 work area；學完內表後，講義 7 第 3 節第一次用到並詳細說明）。

4. CONSTANTS 常數

宣告後不可改值的資料物件，**必須**給 `VALUE`：

```
CONSTANTS gc_max_score TYPE i VALUE 100.
CONSTANTS gc_pass_line TYPE i VALUE 60.
```

```
IF gv_score >= gc_pass_line.
  WRITE / '及格'.
ENDIF.
```

用途：把散落在程式裡的「魔術數字」「魔術字串」集中命名。改規則時只改一處，而且 `gc_pass_line` 比裸寫 `60` 可讀得多。對常數賦值會直接編譯錯誤，這是保護不是限制。

5. 賦值、CLEAR 與型別轉換

```
gv_count = 42.                  " 賦值就是等號
gv_name  = 'HELLO'.
CLEAR gv_count.                 " 恢復成該型別的初始值（i 是 0，c 是空白）
```

不同型別互相賦值時，ABAP 會**自動轉換**（依固定的轉換規則），這很方便也很危險：

```
DATA: gv_char TYPE c LENGTH 3 VALUE '123',
      gv_int  TYPE i.
gv_int = gv_char.               " 自動 '123' → 123，成功
gv_char = 'ABC'.
* gv_int = gv_char.             " 執行期當掉（CONVT_NO_NUMBER）—內容不是數字
```

> 鐵律：轉換錯誤是**執行期**才爆，語法檢查抓不到。來源資料不可信（例如使用者輸入）時，賦值前要自己驗證。

6. 系統欄位初探（structure SY）

系統隨時維護一組欄位，用 `sy-欄位名` 取用，本課程常用的：

欄位	內容
`sy-datum`	今天日期（伺服器）
`sy-uzeit`	現在時間
`sy-uname`	登入帳號
`sy-subrc`	上一個操作的回傳碼（0 = 成功），講義 4 起天天用
`sy-tabix`	內表迴圈目前筆數（講義 4）
`sy-index`	`DO` 迴圈目前圈數

7. 常見錯誤與陷阱

症狀	原因
文字被截成 1 個字	`TYPE c` 沒給 LENGTH
金額加總出現 0.0000001 誤差	用了 `f`，該用 `p DECIMALS`

症狀	原因
對 CONSTANTS 賦值編譯錯誤	常數本來就不可改——改宣告的 VALUE
執行期 CONVT_NO_NUMBER 當掉	字元轉數值但內容不是數字
`LIKE` 後面接型別名報錯	`LIKE` 只能接資料物件，接型別要用 `TYPE`

8. 課堂練習

完成 ex02：宣告各型別變數、用 LIKE 參考、定義常數並觀察轉換行為。